ENTERPRISE SYSTEMS INTEGRATION

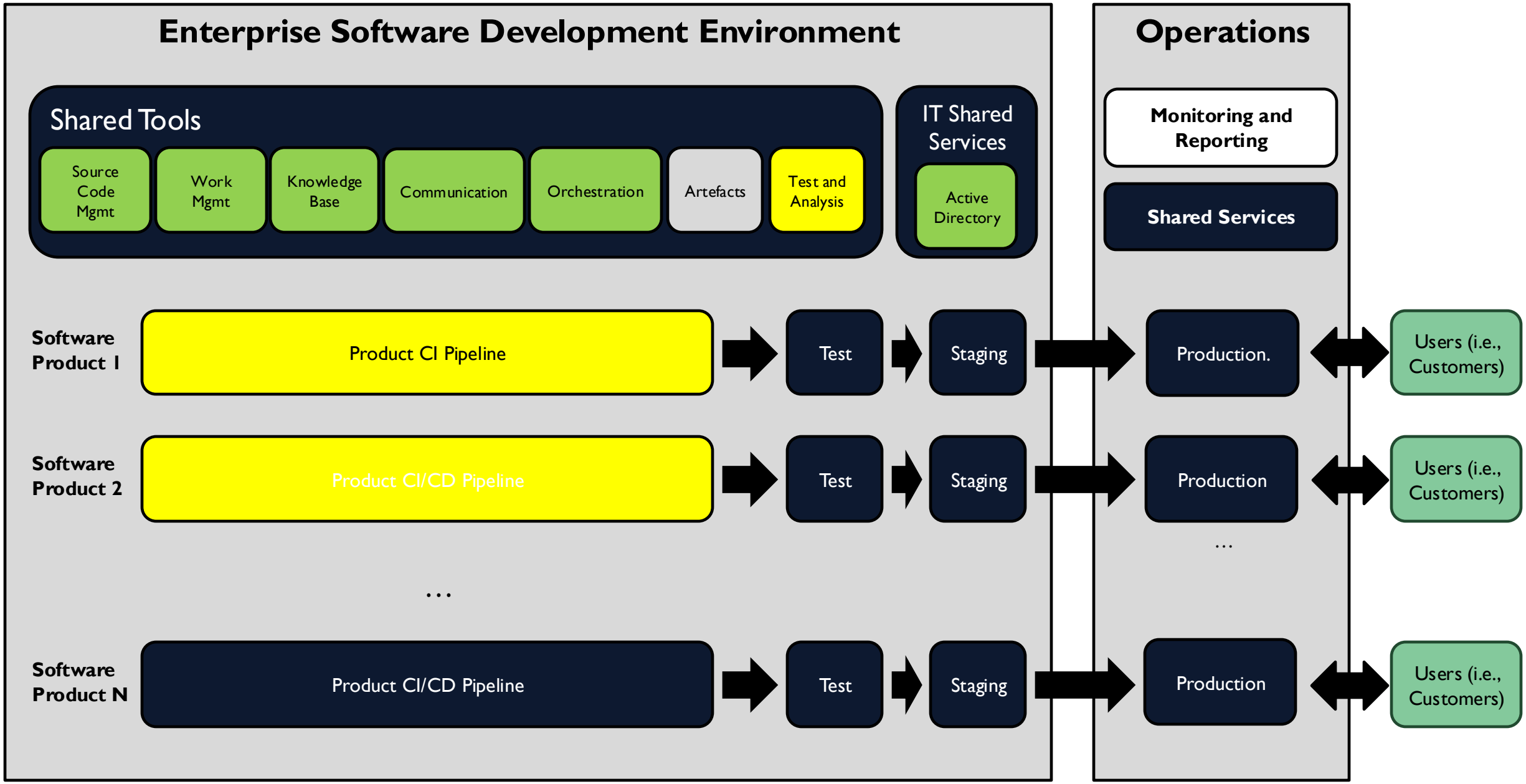
ACIT4850 – WINTER 2026

AGENDA – LESSON 7

- Midterm Preview
- Topics
  - Static Code Analysis
  - SonarQube
- Quiz 6 on D2L
- Midterm Review
- Lab
  - Demo of Lab 6
  - Start on Lab 7

STATIC ANALYSIS

- Code Analysis Without Executing the Program
- A Code Review (i.e., Visual Inspection) is Static Analysis
  - Making sure code complies with project or industry standards
  - Looking for common bugs
- Static Analysis Can Be Done With Automated Tools
  - Can find violation of standards (i.e., naming, structure)
  - Can find common bugs and common signs of poor design/structure
  - Has a harder time with semantic (i.e., logic errors)


Enterprise Software Development Environment
Shared Tools
Source Code Mgmt
Work Mgmt
Knowledge Base
Communication
Orchestration
Artefacts
Test and Analysis
IT Shared Services
Active Directory
Software Product 1
Product CI Pipeline
Test
Staging
Software Product 2
Product CI/CD Pipeline
Test
Staging
...
Software Product N
Product CI/CD Pipeline
Test
Staging
Operations
Monitoring and Reporting
Shared Services
Production.
Production
...
Production
Users (i.e., Customers)
Users (i.e., Customers)
Users (i.e., Customers)

THE ROADMAP (AKA COURSE SCHEDULE)

Week	Topics	Notes
1	• Components of an Enterprise Development Environment • Software Source Code Management	Lab 1
2	• Work Management and Knowledge Base Tools	Lab 2, Quiz 1
3	• Tool Selection – Requirements • Integration and Security	Lab 3, Quiz 2
4	• Tool Selection – Stakeholders/Process • Continuous Integration (CI) Tool • CI Tool Setup	Lab 4, Quiz 3
5	• CI Pipelines – Python	Lab 5, Quiz 4,Assignment 1 Due
6	• CI Pipelines – Shared Libraries	Lab 6, Quiz 5
7	• CI Pipelines – Java and Static Code Analysis	Lab 7, Quiz 6
8	**Midterm Exam**	
9	• CI Pipelines – Alternate Tools	Lab 8, Quiz 7
10	• CI Pipelines – Artifact Management (Java)	Lab 9, Quiz 8, Assignment 2 Due
11	**Computing Reading Week**	
12	• Continuous Delivery (CD) • CD Pipelines - Containerization	Lab 10, Quiz 9
13	• CD Pipelines – Deployment 1 • Developer Workflows with Jenkins	Lab 11, Quiz 10
14	• CD Pipelines – Deployment 2 • Developer Workflows with GitLab CI	Lab 12, Quiz 11, Assignment 3 Due
15	• Final Exam Preview	Lab 12/Assignment 3 Demos in Class
16	**Final Exam**	

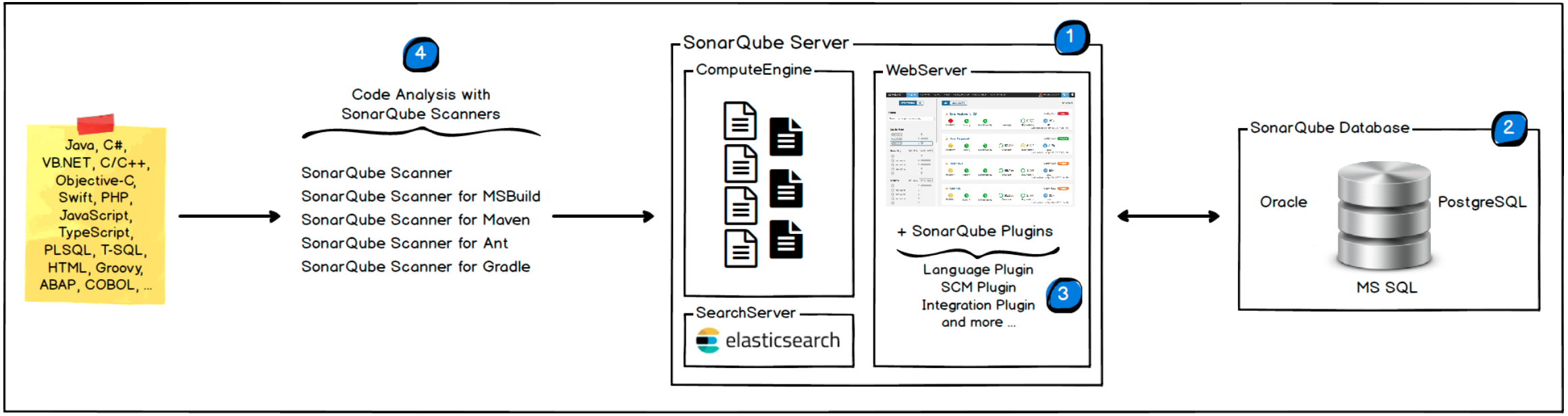
MIDTERM

- Thursday, Feb. 26th at 1pm
- 2 hours
- Covers topics from Week 1 to Week 7
- General Questions – MC, MS, T/F, Written
  - Closed Book, On Paper
  - Some questions will be similar to the quizzes
  - May also include questions on a Jenkinsfile (i.e., identify parts, purpose of each part, fill in blanks), tool upgrade/comparison questions (Assn 1), improvements
- Jenkins Pipeline
  - On your laptop
  - You will need your Apache, Jenkins (Controller and at least your Python Agent) and GitLab VMs running
  - You will have to create and run a simple multi-stage pipeline using one of your existing Python repos (point or carlot)

SONARQUBE

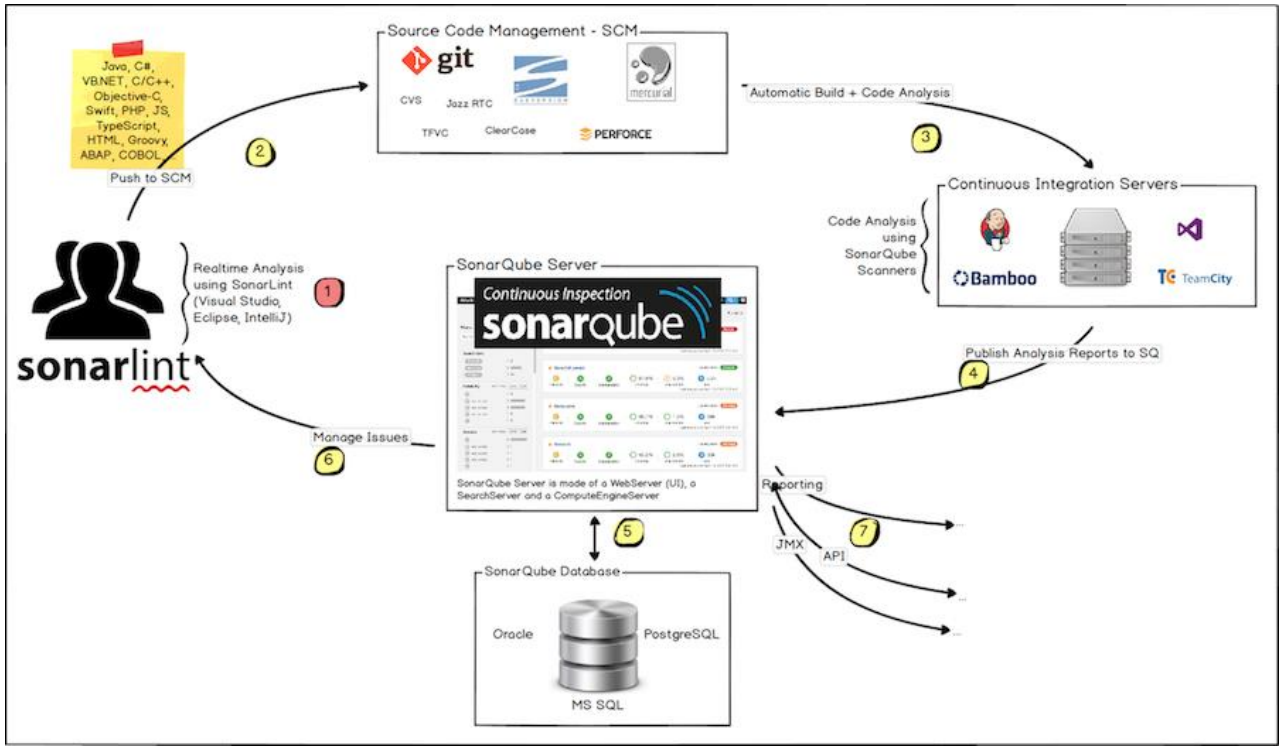
- Last Lab you incorporated a Python Lint tool (pylint). It performed static analysis to find common errors and reported a score out of 10, with a 5 and higher being a pass in our pipeline.
- SonarQube is a "glorified" lint tool with the following features:
  - Scanners to perform the static analysis run on your CI tool
  - Server to receive and analyze the results (i.e., the snapshots) and present them to the user
  - Plugin for integration or language specific processing
  - Database to store the results/analysis for comparison (i.e., the snapshots), custom rules and to record flagged "false positives"
  - Also: IDE Plugins to catch issues even before the CI pipeline

SONARQUBE



Reference: https://docs.sonarqube.org/latest/architecture/architecture-integration/

SONARQUBE



1. SonarLint – Local Analysis in IDE
2. Source Code Management
3. Continuous Integration Server Build where the SonarScanner is run
4. Analysis it sent to the SonarQube Server
5. SonarQube Server processes and stores the results in the SonarQube Database
6. Developers review, comment and challenge their issues in the SonarQube UI
7. Reporting for Managers and APIs/JMX for Ops

Reference: https://docs.sonarqube.org/latest/architecture/architecture-integration/

NEW LAB REQUIREMENTS

- (New) **REQ1180** – The Enterprise Development Environment shall support a Static Code Analysis capability.
- (New) **REQ1190** – The Enterprise Development Environment shall support Continuous Integration Pipelines for Java projects. At minimum, the pipeline will include build, test, analysis, packaging and artifact storage.

NEW LAB REQUIREMENTS

- **REQ1180** – The Enterprise Development Environment shall support a Static Code Analysis capability.

We will use SonarQube for Static Code Analysis in our environment.

We will use a full installation, not a demo instance, so we need a PostgreSQL database to store the analysis and configurations.

NEW LAB REQUIREMENTS

- **REQ1190** – The Enterprise Development Environment shall support Continuous Integration Pipelines for Java projects. At minimum, the pipeline will include build, test, analysis, packaging and artifact storage.

Build, Analysis, Test and Store a simple sample Java application

Build – Using Maven to build the Java JAR file (Java Archive)

Analysis – SonarQube

Test – Run Java jUnit tests

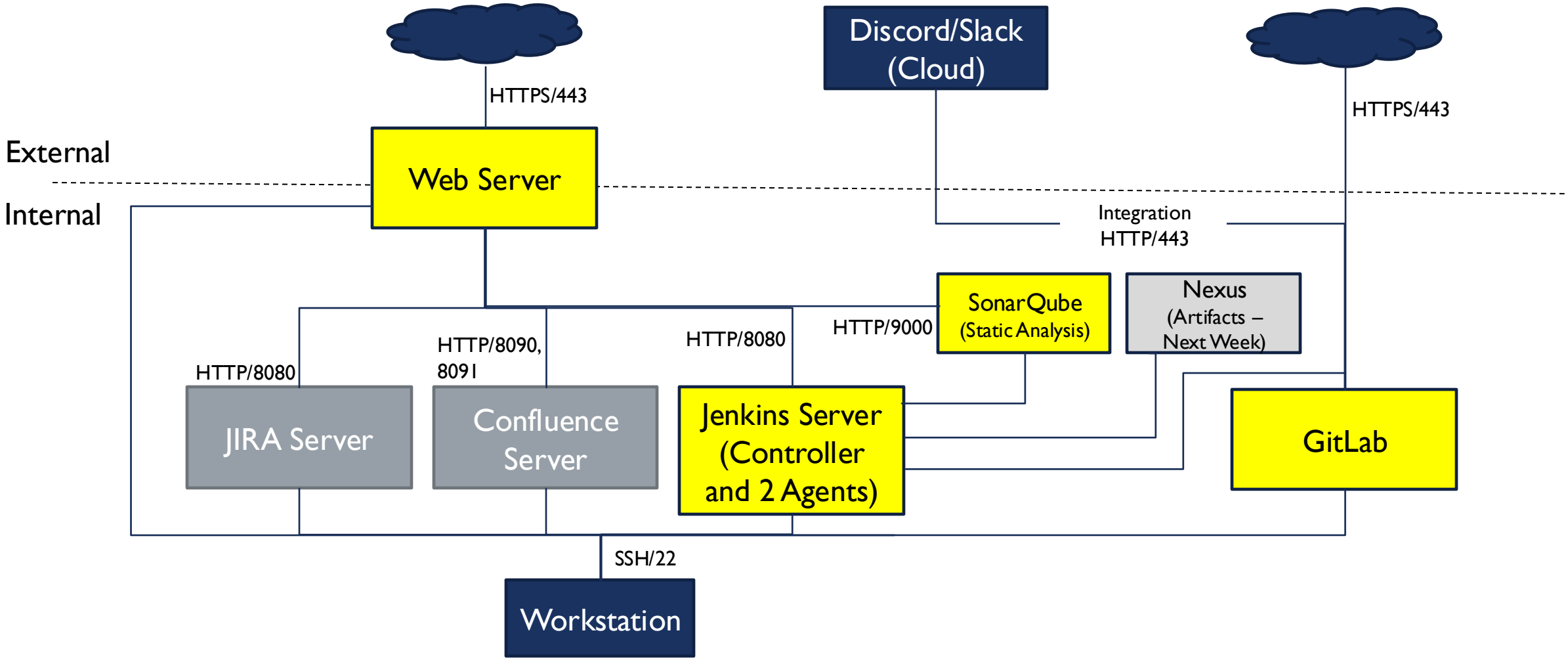
Store – Store the JAR file in the local Maven repository. This is stored in the /root/.m2/repository folder of your Jenkins Docker image. This is basic artifact management for Java (future lab)

Deliver – Today only – it will run the Java JAR file built in the pipeline (simple CD)

DOCKER COMPOSE

- We will be using Docker Compose to run our 3 containers:
  - Jenkins Controller – controller_jenkins
  - Python Agent – agent-python
  - Java Agent – agent-java

YOUR ENTERPRISE DEVELOPMENT ENVIRONMENT (SO FAR)



YOUR LAB 7 DEMO

- Need to simulate:
  - Build Failure
  - Test Failure
  - Code Quality Failure

JAVA HELLO WORLD

App.java

```
package com.mycompany.app;

/**
 * Hello world!
 */
public class App
{

    private final String message = "Hello World!";

    public App() {}

    public static void main(String[] args) {
        System.out.println(new App().getMessage());
    }

    private final String getMessage() {
        return message;
    }

}
```

What would cause:
- A Build Failure
- A Test Failure
- A Code Quality Failure

TestApp.java

```
...
    @Test
    public void testAppConstructor() {
        try {
            new App();
        } catch (Exception e) {
            fail("Construction failed.");
        }
    }

    @Test
    public void testAppMain()
    {
        App.main(null);
        try {
            assertEquals("Hello World!" +
                         System.getProperty("line.separator"),
                         outContent.toString());
        } catch (AssertionError e) {
            fail("\"message\" is not \"Hello World!\"");
        }
    }

...
```

QUIZ 6

- On Static Code Analysis and SonarQube
- On the Learning Hub (aka D2L), Open Book
- Your have 15 minutes to complete it

TODAY'S LAB

1. Demo Lab 6 Before the End of Class
2. Start on Lab 7:
    1. You will do this together with your partner (if applicable)
    2. SonarQube and simple Java Pipeline only.
    3. You must be able to live demo the three scenarios – code failure, test failure and quality gate failure